

Isolation, Protection, and EMI Control in ESP32 Motor Control Systems

Relay vs. Optocoupler vs. Solid-State Relay, Protecting the ESP32, and the Practical Minimum for EMI Control

Report Title	Isolation, Protection, and EMI Control in ESP32 Motor Control Systems
Program	IoT Training Program
Module	Day 13: Isolation, Protection and EMI
Date	4 / 8 / 2026
Prepared By	Malak Alsharqawi
Organization	Cyber Robot

Contents

Section 1: Relay vs. Optocoupler vs. Solid-State Relay	3
1.1 The Big Idea: Two Rooms	3
1.2 The Three Jobs, in Plain Language	4
1.3 The Optocoupler	4
1.4 The Electromechanical Relay	5
1.5 The Solid-State Relay (SSR)	7
1.6 Head-to-Head Comparison Matrix	8
1.7 Choosing the Right Device	8
1.8 Applying This to the Project: L298N Control Lines vs. Motor Power Line	9
1.9 Section 1 Recap	12
Section 2: Protecting the ESP32	14
2.1 Flyback Diodes (Freewheel Diodes)	14
2.2 Optocoupler Isolation Boards	17
2.3 Power Supply Isolation	20
2.4 Section 2 Recap	24
Section 3: EMI, The Practical Minimum	25
3.1 What EMI Is	25
3.2 Where EMI Comes From in This Project	25
3.3 How EMI Travels	26
3.4 Symptoms Worth Recognising	26
3.5 The Three Mitigations That Matter at This Scale	26
3.6 Putting It Together on This Project	28
3.7 Do You Need All of This?	30
3.8 Section 3 Recap	30
References	31

Section 1: Relay vs. Optocoupler vs. Solid-State Relay

Every embedded project eventually reaches the same moment: a microcontroller that runs on a few milliamps has to take charge of something that runs on amps. A 3.3 V pin needs to start a motor, close a valve, or switch a lamp. The pin cannot do that on its own, so a second component has to stand between the two worlds and act on the microcontroller's behalf.

Three components turn up again and again at exactly that boundary: the relay, the optocoupler, and the solid-state relay (SSR). Beginners confuse them constantly, and for a reasonable reason: all three are advertised as providing electrical isolation. That shared feature hides the fact that they are built for three genuinely different jobs. This section explains what each one is, what is physically inside it, how it works step by step, and, most importantly, which one belongs at which point in an ESP32 and L298N motor control circuit.

THE SINGLE SENTENCE TO REMEMBER

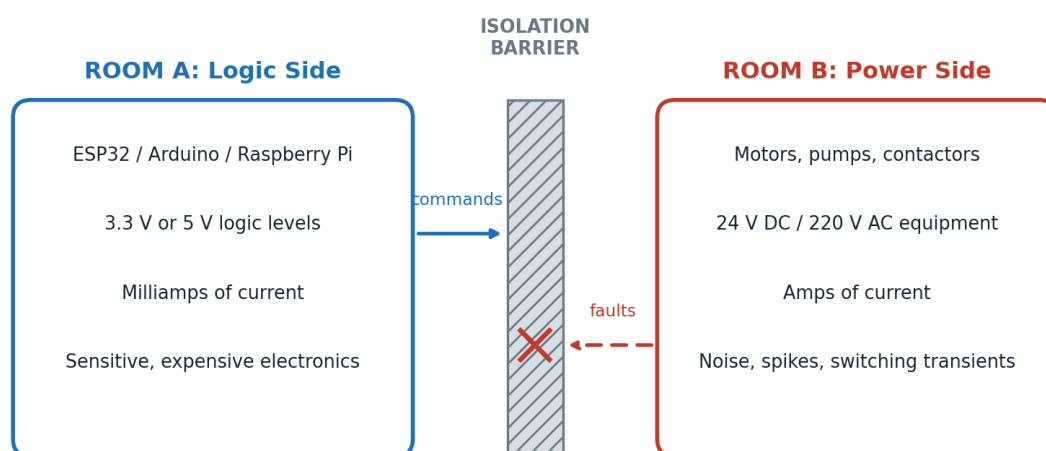
Isolation is a feature, not a purpose. The real difference between these three components is what each one does after the isolation has been provided.

1.1 The Big Idea: Two Rooms

Before looking at any component, it helps to picture the problem as a building with two rooms that must never be allowed to leak into each other.

- **Room A, the logic side.** This is where the ESP32, an Arduino, or a Raspberry Pi lives. Everything here runs at 3.3 V or 5 V, draws milliamps, and is delicate. A single stray voltage spike is enough to destroy a GPIO pin or the whole chip.
- **Room B, the power side.** This is where the motors, pumps, contactors, 24 V industrial equipment, and 220 V AC lamps live. Everything here draws amps, generates its own electrical noise, and is comparatively indestructible.

The engineering goal is simple to state: the command has to get from Room A into Room B, but nothing dangerous may ever travel back the other way. All three components in this section achieve that separation. What they do once the wall is in place is where they differ completely.



All three devices keep the two rooms apart. The difference is what each one does once the barrier is in place.

Figure 1. The two electrical worlds inside a single embedded project, and the barrier that must sit between them.

1.2 The Three Jobs, in Plain Language

A useful way to keep the three components apart in your head is to imagine each one as a different employee, each with a very specific job description.

Component	Job title	What it says
Optocoupler	Messenger	"I will tell the other room what just happened." It transfers information only. It does not switch power.
Relay	Electrician	"I will physically connect or disconnect the power." It switches electricity using a mechanical switch.
Solid-State Relay	Electronic electrician	"I will switch the power too, but electronically." Same job as the relay, no moving parts.

Everything that follows in this section is an elaboration of that table. If you only remember one thing, remember that a messenger cannot carry furniture, and an electrician is a poor choice for delivering a short note.

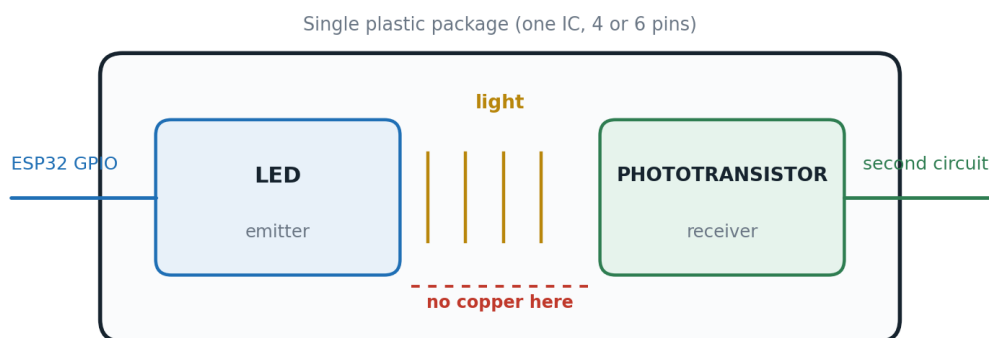
1.3 The Optocoupler

1.3.1 What It Is and What Is Inside It

An optocoupler, also called an optoisolator, transfers a signal from one circuit to another using light instead of a wire. Inside a single small plastic package there are exactly two components:

- **An LED**, on the input side, which lights up when the controlling circuit drives current through it.
- **A phototransistor**, on the output side, which switches on when it detects that light.

The critical detail, and the entire reason the component exists, is that these two halves sit in the same package but are not electrically connected to each other. There is no copper path between them. The only thing that crosses the gap is light.



The signal crosses as light. Electricity has no path from one side to the other.

Figure 2. Inside an optocoupler: an LED and a phototransistor facing each other across a gap that carries light but not current.

1.3.2 How It Works, Step by Step

1. The ESP32 drives its GPIO pin HIGH.
2. Current flows through the internal LED, usually limited by a series resistor.

3. The LED emits light inside the sealed package.
4. The phototransistor on the far side detects that light.
5. The phototransistor turns on.
6. The second circuit sees the same logic level the ESP32 intended to send.

Notice what did not happen anywhere in those six steps: no electrical current crossed from one circuit into the other. Only light crossed the barrier. The information arrived intact, and the electrical connection was never made.

1.3.3 What an Optocoupler Actually Does

An optocoupler copies a signal safely. That is the whole job. Written as a chain, it looks like this: ESP32 HIGH, then optocoupler, then PLC input HIGH. Nothing more is happening.

It is equally important to be clear about what it does not do. An optocoupler does not supply power. It does not drive motors. It does not switch lamps. All it says to the other side is: “the input just became HIGH.”

IN PLAIN TERMS

A telephone lets you communicate with someone in another house. It does not move their furniture for them.

An optocoupler communicates. It does not control large electrical loads.

1.3.4 Typical Applications

- Reading a 24 V industrial sensor with a microcontroller that only tolerates 3.3 V.
- Isolated UART communication between two boards on different power systems.
- Interfacing a microcontroller to a PLC input card.
- Protecting a microcontroller sitting next to electrically harsh equipment.
- Eliminating ground loops between two systems that would otherwise fight over a common 0 V reference.
- Triggering TRIACs in AC control circuits.

1.4 The Electromechanical Relay

1.4.1 What It Is and What Is Inside It

A relay is an electrically controlled mechanical switch. Instead of a finger pressing a switch, a small electric current does the pressing. Inside every conventional relay there are three parts that matter:

- **A coil**, a winding of fine wire that becomes an electromagnet when current flows through it.
- **An iron armature**, a small metal arm that the electromagnet can pull.
- **Mechanical contacts**, a pair of metal surfaces that touch when the armature moves, completing the load circuit.

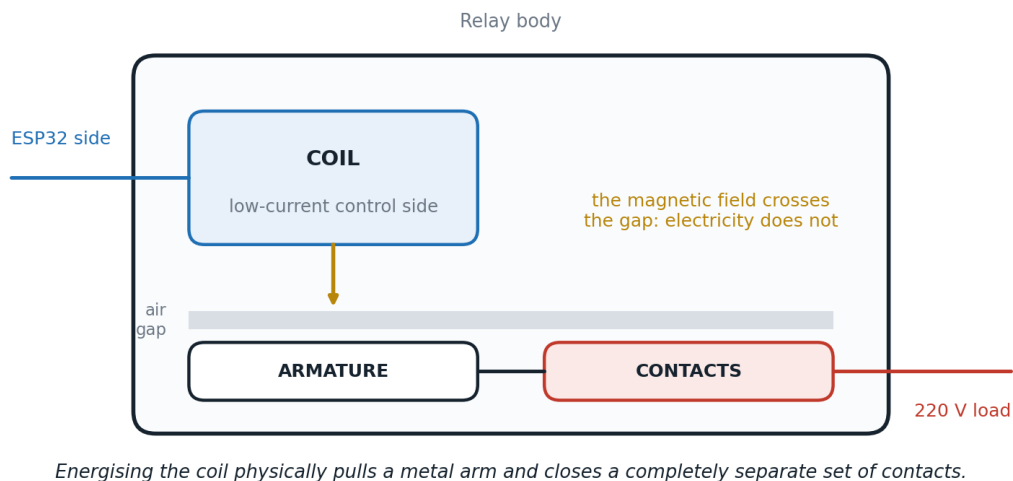


Figure 3. Inside a relay: a low-current coil, an air gap, and a completely separate set of high-current contacts.

1.4.2 How It Works, Step by Step

7. The ESP32 drives its GPIO pin HIGH (in practice, through a transistor or a ready-made relay module).
8. Current flows through the relay coil.
9. The coil creates a magnetic field.
10. The magnetic field pulls the metal armature.
11. The contacts close.
12. The external load, on its own separate circuit, receives power.

Unlike an optocoupler, a relay genuinely connects power to a load. Without a relay, wiring an ESP32 GPIO pin directly to a 220 V lamp is not merely a bad idea, it is impossible: the pin cannot supply that voltage or that current. With a relay in the path, the ESP32 issues a low-power command and the lamp turns on from its own mains supply.

1.4.3 Why a Relay Provides Isolation

The coil and the contacts never touch each other. They are separated by a physical air gap inside the relay body. The magnetic field crosses that gap; electricity does not. That is why the ESP32 remains electrically isolated from the high-voltage load even while it is commanding it, and it is the reason a relay can safely bridge a 3.3 V logic pin and a 220 V mains circuit.

1.4.4 Strengths, Weaknesses, and Typical Applications

Advantages	Disadvantages
Cheap and widely available	Mechanical wear: a finite number of switching cycles
Can switch both AC and DC loads	Audible click on every operation
Handles large currents	Slow switching, in the region of 5 to 15 ms
Excellent, well-understood galvanic isolation	Completely unsuitable for PWM

Typical applications include home automation, pumps, fans, industrial equipment, AC lighting, and high-current DC loads. In other words, anywhere a load has to be turned fully on or fully off, and not very often.

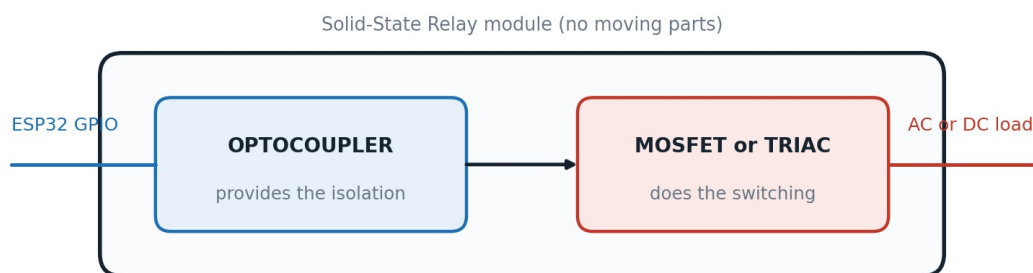
1.5 The Solid-State Relay (SSR)

1.5.1 What It Is and What Is Inside It

An SSR performs exactly the same job as a relay: it switches electrical power. The difference is that it has no moving parts. Instead of mechanical contacts, it uses semiconductors. A typical SSR contains:

- An **optocoupler**, which provides the isolation between the control side and the load side.
- A **MOSFET** in a DC SSR, or a **TRIAC** in an AC SSR, which does the actual switching.

This is worth stating explicitly, because it resolves most of the confusion between the three devices: an SSR is essentially an optocoupler bolted onto an electronic power switch, sold as one module. The optocoupler alone could never drive the load. The optocoupler plus the power stage can.



An SSR is simply an optocoupler bolted to an electronic power switch, in one package.

Figure 4. Inside a solid-state relay: an optocoupler for isolation, followed by a MOSFET or TRIAC that carries the load current.

1.5.2 How It Works, Step by Step

13. The ESP32 drives its GPIO pin HIGH.
14. The internal optocoupler activates.
15. The MOSFET or TRIAC turns on.
16. Current flows to the load, with no moving contacts and no clicking.

1.5.3 Relay vs. SSR

Both perform the same function and both turn electrical loads on and off. The only real difference is how they switch. A relay switches with mechanical contacts, which is why you hear a click. An SSR switches electronically, which is silent. The consequences of that difference are practical rather than theoretical: no moving parts means no mechanical wear, so an SSR can perform millions of switching cycles and can switch far faster than a relay ever could.

THE THREE-QUESTION TEST

Can an optocoupler turn on a lamp? No. The optocoupler delivers a HIGH signal, but the lamp still has no power, because nothing connects it to the electrical supply.

Can a relay turn on a lamp? Yes. The relay physically connects the 220 V supply to the lamp.

Can an SSR turn on a lamp? Yes. The SSR connects the same supply electronically instead of mechanically.

1.6 Head-to-Head Comparison Matrix

The table below places the three devices side by side across the attributes that actually decide a design: whether the device switches power at all, whether it can survive PWM, what kind of isolation it provides, how much current it can carry, and whether an ESP32 pin can drive it without help.

Attribute	Optocoupler	Relay	Solid-State Relay
Primary job	Transfer signals	Switch power	Switch power
Provides electrical isolation	Yes	Yes	Yes
Isolation type	Optical (light across a gap)	Physical air gap plus magnetic coupling	Optical, inside the module
Switches electrical power	No	Yes	Yes
Can drive a motor directly	No	Yes	Yes
Internal technology	LED plus phototransistor	Coil, armature, mechanical contacts	Optocoupler plus MOSFET or TRIAC
Moving parts	No	Yes	No
Audible click	No	Yes	No
Mechanical wear	None	Yes, finite contact life	None
PWM capable	No (signal level only, not a power path)	No (contacts cannot follow kHz switching)	Partly: DC SSRs generally yes, AC SSRs usually not for high-frequency PWM
Typical switching speed	Microseconds	5 to 15 ms	Microseconds to milliseconds
Typical current capability	Signal level only	High	Moderate to high
Typical voltage handled	Logic signals	AC and DC loads	AC or DC, model dependent
ESP32 direct drive	Usually yes, with a series resistor	Usually needs a transistor or driver, unless a relay module is used	Usually yes, if the input current stays within GPIO limits

READING THE MATRIX

Three rows carry almost all of the decision weight:

Switches electrical power separates the messenger from the two electricians. **PWM capable** rules both electricians out of any speed-control role. **ESP32 direct drive** tells you whether you need extra driver hardware in between.

1.7 Choosing the Right Device

Reduced to a lookup table, the decision is genuinely this short:

I want to...	Use
Transfer a digital signal safely	Optocoupler
Protect the ESP32 from a high-voltage environment	Optocoupler
Read a 24 V industrial sensor	Optocoupler
Turn a lamp on and off	Relay or SSR
Control a pump	Relay or SSR
Control a fan	Relay or SSR
Switch silently	SSR
Switch as cheaply as possible	Relay
Perform millions of switching cycles	SSR

THE EASY WAY TO REMEMBER IT

Ask yourself one question: “Am I sending information, or am I switching power?”

If you are sending information, use an optocoupler. If you are switching electrical power, use a relay or an SSR.

1.8 Applying This to the Project: L298N Control Lines vs. Motor Power Line

This is where the theory earns its keep. Beginners building an ESP32 and L298N motor project ask a predictable set of questions:

- Why does the L298N not need an optocoupler?
- Why would I ever use a relay in this project?
- Why can I not put a relay on the Enable (ENA) pin?
- If the ESP32 is only sending a signal, why would I isolate it?
- Since relays, SSRs, and optocouplers all isolate circuits, are they not doing the same thing?

Every one of those questions dissolves the moment you accept that a motor control circuit contains two completely different types of connection: control signals, which carry information, and power lines, which carry energy. They serve different purposes and they need different components.

1.8.1 The Two Layers of a Motor Control System

Think of the system as two independent layers stacked on top of each other.

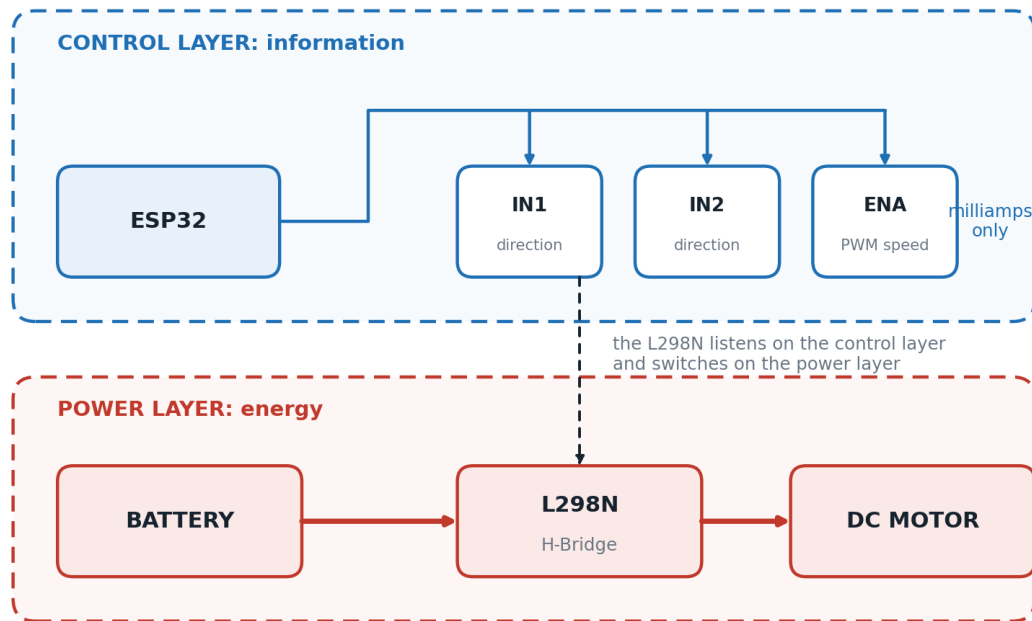


Figure 5. The control layer carries information at milliamp levels. The power layer carries energy at amp levels. The L298N is the only part that lives in both.

The single most important observation in this whole section is visible in that diagram: the ESP32 never powers the motor. The ESP32 sends commands. The L298N supplies power to the motor. These are completely different jobs, performed by different components, on different wires.

1.8.2 The Control Layer: IN1, IN2, and ENA

The control pins are IN1, IN2, and ENA. They carry information, not power. The messages they carry are things like “go forward”, “go backward”, “stop”, and “run at 50 percent speed”. Those are commands, nothing more.

When the ESP32 drives IN1 HIGH, almost nothing happens electrically. A tiny current enters the L298N input, and the L298N receives the instruction “rotate forward”. No motor current flows through IN1 at any point. This is why the common phrase “the ESP32 controls the motor” is not quite right. The ESP32 controls the L298N; the L298N controls the motor. The actual switching happens inside the L298N’s H-Bridge, using the current from the motor’s own power supply.

The L298N was specifically designed to be driven by a microcontroller. It expects logic inputs: HIGH, LOW, and PWM. Connecting an ESP32 GPIO pin straight to L298N IN1 is completely normal practice. In a typical hobby build the two devices share the same ground, use compatible logic levels, and sit in the same relatively clean electrical environment, so no additional isolation is required. That is why almost every ESP32 and L298N project on the internet wires them together directly.

WOULD AN OPTOCOUPLER WORK ON IN1?

Yes, and the L298N would still receive HIGH and LOW perfectly. But it is worth being precise about what the optocoupler would be doing: it is not helping the L298N understand the signal, because the L298N already understands the signal perfectly. The only thing an optocoupler changes is how the signal travels. Instead of electricity crossing the wire, light crosses a gap.

The reason to add one is therefore never better motor control. It is protection of the ESP32. That case is made in full in Section 2.2.

1.8.3 Why Not Put a Relay on IN1 or IN2?

Technically, it would work. A relay could connect IN1 to HIGH or to LOW, and the L298N would obey. But it would be solving a problem that does not exist, because the L298N already accepts logic signals directly. Adding a relay to a logic line only makes the system slower, noisier, larger, and more expensive, with no benefit whatsoever.

1.8.4 Why a Relay Absolutely Cannot Control ENA

The ENA pin is the speed control input, and it normally receives PWM. PWM works by rapidly switching the signal on and off, and the ratio of on-time to off-time sets the effective speed, from 0 percent through 20, 40, 80, and up to 100 percent.

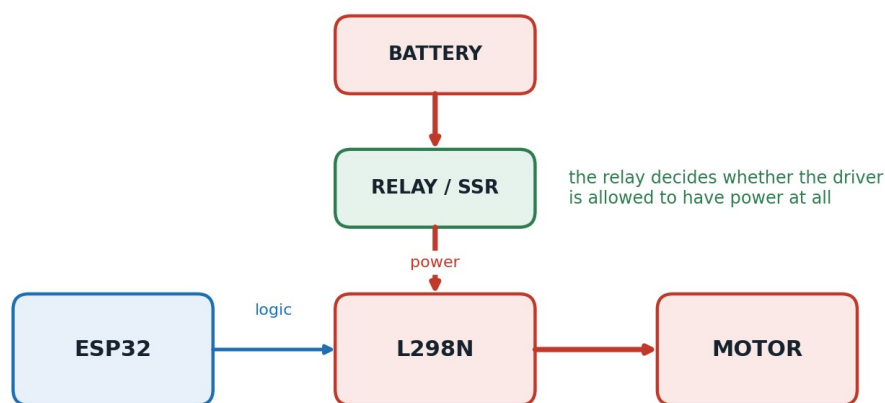
A typical PWM frequency in this kind of project is around 5 kHz. That means 5,000 on and off cycles every second. No electromechanical relay can switch at that rate. Attempting it would make the relay click continuously, wear out within minutes, and fail mechanically. The transistors inside the L298N exist precisely because this kind of fast switching needs semiconductors, not moving metal.

1.8.5 The Power Layer: The Battery Feed and the Emergency Stop

Now look at a different wire entirely: the one running from the battery, into the L298N, and out to the motor. This wire carries motor current, motor voltage, and real electrical power. Unlike IN1 or ENA, this is not information. This is energy. That distinction is what decides which component belongs here.

Consider a realistic failure. The software crashes. The ESP32 freezes with its outputs stuck in whatever state they were in. The L298N, which is only obeying the last instruction it received, keeps running the motor. You need a way to guarantee the motor stops that does not depend on the software working.

The answer is to put a relay or an SSR in the battery feed, upstream of the L298N. When it opens, no power reaches the driver, no power reaches the motor, and the software can no longer move the motor even by accident. Relays and SSRs are the right choice here because they are designed to interrupt power rather than logic signals, and they safely switch high voltage, high current, and large electrical loads. That is their entire job.



Logic path in blue, power path in red. A relay or SSR belongs only on the red path.

Figure 6. A relay or SSR sits in the power path, upstream of the driver. It never belongs on the logic path.

WHY CAN AN OPTOCOUPLER NOT DO THIS?

Because an optocoupler only transfers tiny signals. It cannot carry motor current, it cannot switch power, and it was never designed to. Wiring a battery through an optocoupler into a motor is not a compromise, it is simply impossible.

1.8.6 But Does the L298N Not Already Switch the Motor?

This is the last common misunderstanding, and it is a fair one. The L298N does already switch the motor, so why add a relay at all?

Because they operate at different levels of control. The L298N decides forward, reverse, brake, and speed. The relay decides something more fundamental: should the L298N have power at all? Think of a computer. The operating system can turn the monitor black. Unplugging the monitor's power cable is something completely different. The relay is the equivalent of unplugging the cable.

1.8.7 Circuit-Point Decision Table

Circuit part	Carries	Purpose	Recommended device	Why
IN1	Logic signal	Direction command	ESP32 directly, or an optocoupler if isolation is needed	Only a tiny control current is required
IN2	Logic signal	Direction command	ESP32 directly, or an optocoupler if isolation is needed	Same as IN1
ENA	PWM logic signal	Speed control	ESP32 directly	PWM requires thousands of fast transitions per second
Battery to L298N	Power	Supplies energy to the driver	Relay or SSR	Safely disconnects the driver from its power source
L298N to motor	High-current power	Drives the motor	Controlled internally by the L298N H-Bridge	The L298N is specifically designed to switch this power

1.8.8 Never-Use Rules

Stated as two short rules that are worth memorising:

- **For IN1, IN2, and ENA:** use the ESP32 directly, or an optocoupler if electrical isolation is genuinely required. Never use a relay or an SSR.
- **For the battery supply:** use a relay or an SSR. Never use an optocoupler alone.

1.9 Section 1 Recap

- All three devices isolate two circuits, but isolation is a shared feature, not their purpose.
- An optocoupler transfers information using light. It cannot supply or switch power.
- A relay transfers electrical power using a mechanical switch, isolated by a physical air gap.
- An SSR transfers electrical power using semiconductors, isolated by an internal optocoupler.

- A motor control circuit has two layers: a control layer carrying information and a power layer carrying energy.
- IN1, IN2, and ENA belong to the control layer, so they take a direct GPIO connection or an optocoupler.
- The battery feed belongs to the power layer, so it takes a relay or an SSR, typically as an emergency stop.
- A relay can never drive ENA, because PWM at roughly 5 kHz means 5,000 switching events per second.

GOLDEN RULE

Always ask yourself: “Am I sending a command, or switching power?” If you are sending a command, use the logic path: a direct GPIO connection, or an optocoupler when isolation is required. If you are switching electrical energy, use a power-switching device such as a relay or an SSR.

Section 2: Protecting the ESP32

Section 1 answered the question of which component belongs where. This section answers a harder one: what actually goes wrong if you get it wrong, and what a designer puts in place to stop it. Three protection measures cover the overwhelming majority of real failures in an ESP32 and L298N project, and they are treated here in the order in which a beginner normally meets them: flyback diodes, optocoupler isolation boards, and power supply isolation.

These three are not alternatives to each other. They defend against three different mechanisms. A flyback diode handles energy stored in a coil. An optocoupler handles a conductive path that should not exist. Power isolation handles current and noise travelling along a shared supply. Getting one right does not excuse skipping the others.

2.1 Flyback Diodes (Freewheel Diodes)

Whenever you control an inductive load, meaning a relay coil, a DC motor, a solenoid, or an electromagnet, you will almost always encounter a flyback diode. It is also called a freewheel diode, a kickback diode, or a snubber diode, and the four names all describe the same component doing the same job.

At first sight the wiring looks wrong. The diode is connected across the component rather than in series with it, which is the opposite of what beginners expect. The reason becomes obvious once you understand how inductors behave.

2.1.1 The Rule Everything Else Follows

THE IMPORTANT RULE

An inductor resists changes in current.

This is one of the fundamental laws of electromagnetics, and every part of this subsection follows from it.

When current begins flowing through an inductor, the inductor stores energy in a magnetic field. When that current is suddenly interrupted, the magnetic field collapses, and the energy it was holding has to go somewhere. If there is no safe path available, the inductor will generate whatever voltage is required to keep the current flowing, and that voltage can be enormous. This phenomenon is called inductive kickback, or back EMF.

2.1.2 What Happens When the Switch Opens

Follow a relay coil controlled by a transistor through three stages.

Step 1, current flows normally. The transistor turns on. Current flows from the 5 V rail, through the coil, through the transistor, to ground. The coil creates a magnetic field. Everything is calm.

Step 2, the transistor suddenly turns off. The current now has nowhere to go. But the inductor hates sudden changes in current, so the magnetic field immediately starts collapsing and the stored energy tries to keep the current flowing.

Step 3, the voltage spike. The inductor generates whatever voltage is necessary to continue that current. Instead of the 5 V that was there a moment ago, it might produce 30 V, 50 V, 100 V, or even several hundred volts for a very short time. That spike appears across the coil, and therefore across the switching device connected to it.

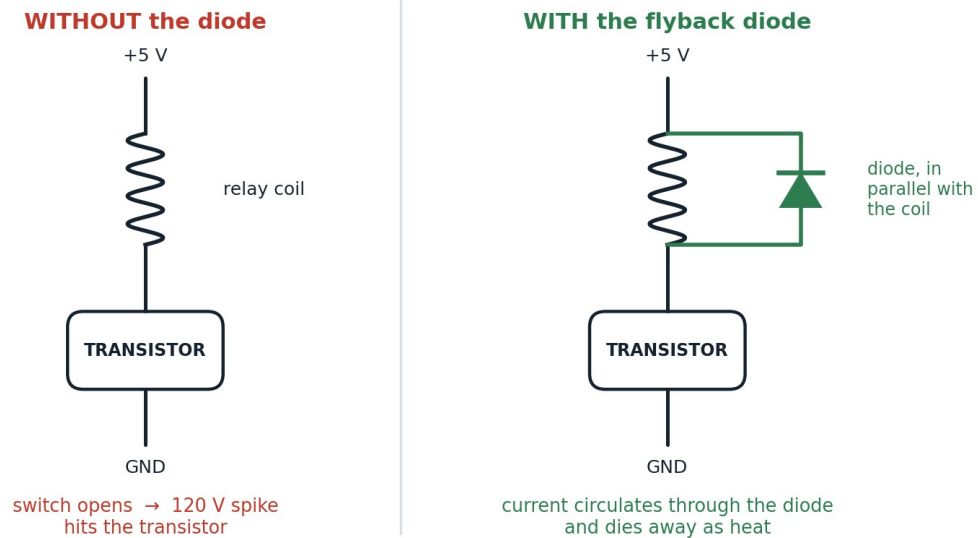


Figure 7. The same relay coil circuit without and with a flyback diode. The diode sits in parallel with the coil, not in series with it.

2.1.3 Why the Spike Is Dangerous

Suppose the transistor is rated for 40 V, which would be a perfectly sensible choice for a 5 V coil. The collapsing magnetic field generates 120 V. The transistor now experiences 120 V where the design expected 5 V. The possible results are:

- A damaged transistor.
- A damaged MOSFET.
- A damaged driver IC.
- A damaged microcontroller, indirectly, through whatever the failed part is connected to.
- Electrical noise injected throughout the rest of the circuit.

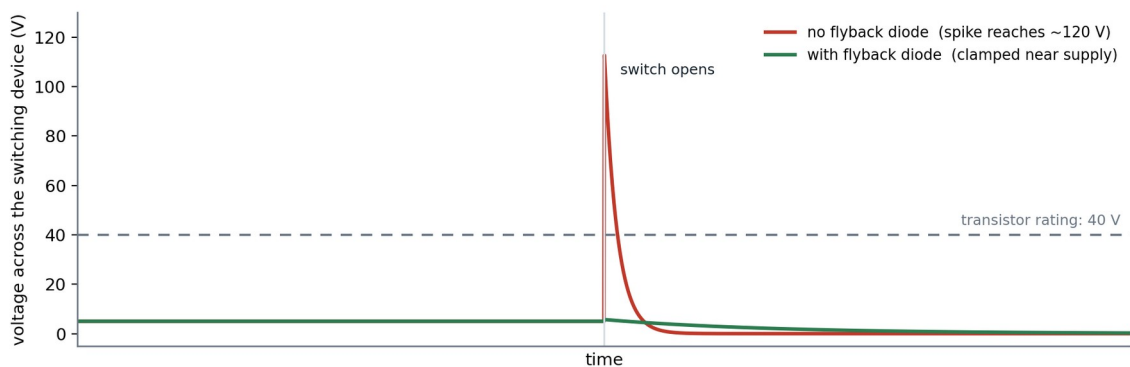


Figure 8. Voltage across the switching device at the moment the coil is de-energised, with and without flyback protection.

2.1.4 The Solution and Where the Diode Goes

A flyback diode gives the current a safe path after the switch turns off. Instead of producing a huge voltage spike, the current simply circulates through the diode until the stored energy is dissipated as heat in the coil's own resistance and in the diode.

The diode is connected across the inductive load, in parallel with it, and not across the transistor. That placement is not a matter of preference. The stored magnetic energy is generated in the inductor, so the safe path has to be offered at the inductor.

2.1.5 Why the Diode Does Nothing During Normal Operation

The diode is connected in reverse bias while the circuit is running normally, so it sees a reverse voltage, stays off, and has no effect at all on ordinary operation.

When the switch turns off, the magnetic field collapses and the polarity across the coil reverses. Suddenly the diode is forward biased, and current circulates in a loop through the coil and the diode until the energy is gone. This is also where the name comes from: when the field collapses, the current “flies back” into the coil through the diode instead of trying to jump across the switching transistor.

2.1.6 Flyback on a Relay Coil and on a DC Motor

The relay coil is the classic case and the most common application. Without the diode you get a large voltage spike; with it you get a safe current path.

A DC motor is also an inductive load. Its windings behave like a large inductor, so when power is removed the motor also generates back EMF. Where a motor is switched in only one direction, for example by a single MOSFET, a flyback diode across the motor protects the circuit in exactly the same way.

2.1.7 What About the L298N?

The L298N is an H-Bridge motor driver, and it behaves differently from a simple transistor because it constantly switches current in both directions. A single diode cannot handle flyback current flowing both ways, so multiple diodes are required, arranged around the bridge.

The practical consequence is a relief: most L298N modules already include these protection diodes on the board, and internally they provide safe paths for the motor current whenever the transistors switch. This is why you normally do not add an external flyback diode across the motor when using a standard L298N module. If you add a relay to the design, however, that relay’s coil is a new inductive load and it does need its own protection.

2.1.8 Why Not Put the Diode Somewhere Else?

Beginners often try placing the diode across the transistor, across the power supply, or across the ESP32. None of those locations properly protects anything, because none of them is where the stored magnetic energy lives. The diode must be connected directly across the inductor.

2.1.9 Which Loads Need Flyback Protection

Device	Needs a flyback diode?	Why
Relay coil	Yes	The coil stores magnetic energy
Solenoid	Yes	Large inductive coil
Electromagnet	Yes	Inductive load by definition
DC motor, single-direction switching	Yes	Motor windings are inductive
Stepper motor	Yes, normally handled by the driver IC	Multiple inductive coils
Servo motor	Usually no, handled internally	The driver electronics already include protection
L298N motor output	Usually no external diode	Protection diodes are built into most modules

2.1.10 Choosing the Diode

The diode has to be able to handle the coil current and the reverse voltage of the circuit it sits in. Common choices are:

- **1N4148**, for small signal relays.
- **1N4001 to 1N4007**, for general-purpose relay protection.
- **Schottky diodes**, where faster switching matters.

2.1.11 A Common Misconception

Many beginners assume that “the diode protects the relay”. Not exactly. The relay is usually fine. The diode protects the switching device: the transistor, the MOSFET, the driver IC, and indirectly the microcontroller, from the voltage spike that the relay coil or motor generates.

Question	Answer
What causes the voltage spike?	The collapsing magnetic field in an inductor
What does a flyback diode do?	Provides a safe path for the current after the switch turns off
Where is it connected?	In parallel with the inductive load: relay coil, solenoid, or motor winding
Why is it reverse biased normally?	So that it does not affect normal circuit operation
When does it conduct?	Immediately after the switching device turns off and the inductor's polarity reverses
Does an L298N module usually need an external one?	No. Most L298N modules already include the required protection diodes

GOLDEN RULE

Whenever you switch an inductive load, ask: “where will the stored magnetic energy go when I turn it off?” If the answer is not obvious, you probably need flyback protection.

2.2 Optocoupler Isolation Boards

In embedded systems, and particularly in motor control, you will often see an optocoupler isolation board placed between the microcontroller and the motor driver. The obvious question is why. If the ESP32 is only sending signals and never receiving them, what is there to isolate?

THE ANSWER, IN ONE LINE

Signals travel in one direction, but electrical faults travel in both.

The purpose of an isolation board is not to improve communication. Its purpose is to prevent electrical energy from crossing between two different electrical systems.

2.2.1 The Two Electrical Worlds

A typical motor control system contains two environments that have almost nothing in common, joined by a wire.

Logic side	Power side
Low voltage, 3.3 V	High current
Low current	Motors and relays
Sensitive electronics	Switching transients
Microcontroller GPIO pins	Electrical noise and voltage spikes

The goal of the whole exercise is to keep those two worlds separated.

2.2.2 What Happens Without Isolation

A normal connection is simply an ESP32 GPIO pin wired to IN1 on the motor driver, plus a common ground. During normal operation everything works exactly as intended: the ESP32 outputs HIGH, the driver receives the command, and the motor moves.

The problem is what that wire actually is. It is a piece of copper. Copper does not know which device is the sender. If something abnormal happens on the motor side, electricity can travel backwards along the same wire. Possible causes include voltage spikes, ground bounce, motor back EMF, wiring mistakes, power supply failures, electrical noise, and outright driver failure. Even though the ESP32 is only ever transmitting commands, its GPIO pins remain electrically connected to the motor driver the entire time.

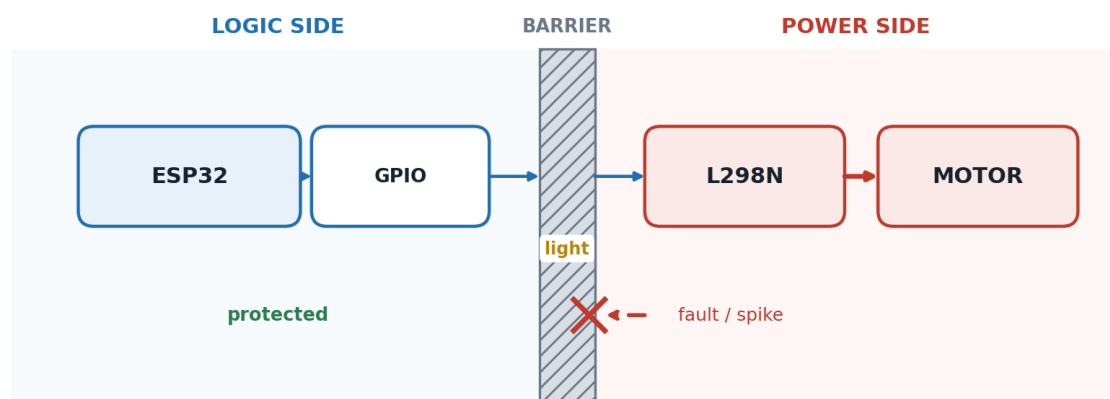
THE CONCEPT THAT FIXES THE CONFUSION

Many beginners imagine GPIO pins as one-way roads. They are not. They are simply electrical connections.

Under normal conditions, information travels from the ESP32 to the driver. During a fault, electrical energy can travel from the driver to the ESP32. The wire allows current to move in either direction, and it has no opinion about which one you intended.

2.2.3 What an Optocoupler Changes

Instead of a wire, an optocoupler uses light. The ESP32 drives an LED, the LED emits light, a phototransistor on the far side detects it, and the motor driver receives the signal. There is no electrical connection between the two sides at any point. Only light crosses the gap. The signal still reaches the motor driver; electricity cannot.



Information crosses the barrier as light. Electrical energy has no path across it at all.

Figure 9. The optocoupler as an isolation barrier: commands cross as light, while a fault on the power side is stopped at the wall.

Consider what this means during a real fault. Imagine a 120 V spike appearing on the motor side. Without isolation, that spike travels along the GPIO wire straight into the ESP32, and the possible damage includes burned GPIO pins, a damaged microcontroller, random resets, software crashes, and permanent hardware failure. With an optocoupler in the path, the spike reaches the optocoupler and stops there. The light path keeps working; the electrical energy does not get across.

2.2.4 Why Not Just Add Protection Components?

Some circuits add resistors, capacitors, TVS diodes, and filters. These genuinely help reduce noise, but they share one limitation: they do not remove the electrical connection. An optocoupler does something fundamentally different. It completely breaks the conductive path.

2.2.5 Ground Isolation

One of the biggest advantages of an isolation board is that the two circuits no longer need to share the same ground. Without isolation, the ESP32 ground and the motor driver ground are physically connected, which can create ground loops, noise coupling, voltage offsets, and communication problems. With isolation, the grounds are completely independent and only the information crosses.

2.2.6 Why It Is Called the Main Barrier

The optocoupler is often described as the main isolation barrier because it is the exact point where electrical continuity stops. Everything before it belongs to the logic side. Everything after it belongs to the power side. That barrier is what prevents dangerous electrical conditions from ever reaching the microcontroller.

2.2.7 Does Every L298N Project Need One?

No, and it is worth being honest about that. For a typical hobby project running an ESP32, an L298N, and a 12 V motor, the ESP32 and the L298N are designed to work together. They usually share a common ground, use compatible logic levels, and operate in a relatively clean electrical environment. Direct GPIO connections are the normal and recommended approach, and adding an isolation board would usually increase cost and complexity without providing significant benefit.

Isolation boards become valuable when the power side becomes electrically harsh. The clear cases are:

- **Industrial automation:** PLC systems, factory machines, and high-power motors.
- **Separate power systems:** independent battery packs, different power supplies, or high-voltage equipment.
- **Long cable runs:** signals travelling tens of metres are far more susceptible to interference, and isolation improves reliability.
- **High electromagnetic noise:** large DC motors, variable frequency drives, welding equipment, and heavy machinery.
- **Expensive controllers:** many industrial designs isolate every GPIO simply because replacing an optocoupler costs far less than replacing the controller.

2.2.8 A Common Misconception

Beginners sometimes believe that the optocoupler helps the driver understand the GPIO signal. This is incorrect. The motor driver already understands HIGH, LOW, and PWM perfectly well. The optocoupler changes how the signal is transferred, not what the signal means. The information stays exactly the same; the electrical connection disappears.

Question	Answer
Why use an optocoupler isolation board?	To electrically isolate the logic side from the motor side
Does it improve the command signal?	No. The logic signal is unchanged
What does it actually protect?	The microcontroller and other sensitive logic electronics
Why is it called the isolation barrier?	It is the point where electrical continuity between the two circuits ends
Can electrical faults cross it?	No. Only light transfers the information
Do hobby L298N projects usually need one?	No. Direct GPIO connections are normally sufficient
When is it recommended?	Industrial systems, separate grounds, noisy environments, high voltages, or long cable runs

FINAL MENTAL MODEL

Imagine two buildings. One is an office full of computers, which is the ESP32. The other is a factory full of heavy machinery, which is the motor driver and motor. The office manager needs to send instructions to the factory.

Method 1, a copper wire. The message travels. But if there is an electrical accident in the factory, it can travel back through the same wire and damage the office.

Method 2, a fibre-optic link, which is what an optocoupler is. The message still arrives, but electrical energy cannot travel through light, so the office stays safe even if something catastrophic happens in the factory.

Golden rule: an isolation board does not exist because the motor driver needs help understanding the signal. It exists because the ESP32 needs protection from the electrical environment on the motor side.

2.3 Power Supply Isolation

One of the most common beginner mistakes is powering the ESP32 and the motor from the exact same power rail without understanding the consequences. A typical beginner circuit takes a 12 V battery, feeds it straight to the L298N and motor, and also feeds it into a voltage regulator that produces 3.3 V for the ESP32.

At first glance this seems perfectly reasonable. There is only one battery, after all. So why do experienced engineers keep recommending that motor power be isolated from microcontroller power?

THE ANSWER

Motors are noisy loads. Microcontrollers require clean, stable power.

The problem is not that they share the same battery. The problem is how the motor behaves while it is running.

2.3.1 Two Very Different Loads

The ESP32 needs stable voltage, low electrical noise, a small current, and generally clean power. A typical ESP32 consumes somewhere between 80 and 250 mA, with short Wi-Fi transmission peaks reaching up to 500 mA, and its supply voltage should stay close to 3.3 V.

A DC motor is a completely different animal. It constantly changes its current draw depending on whether it is starting, stopping, changing speed, carrying a load, or reversing direction. A small motor that normally draws 500 mA may briefly draw 2 to 5 A during startup or when stalled. That sudden demand affects the entire power system, not just the motor branch.

2.3.2 What Happens the Moment the Motor Starts

Picture the motor stopped, drawing 0 A. The ESP32 commands the motor on. Immediately, the motor tries to accelerate, and for a short moment it behaves almost like a short circuit, pulling a large current from the battery. That large current causes the supply voltage to dip.

The reason is Ohm's law, applied to wiring nobody thinks about. No battery or wire is perfect. Every wire, connector, battery, and PCB trace has a small resistance, and when a large current flows through a resistance, a voltage drop appears across it:

WORKED EXAMPLE

Voltage drop = Current × Resistance

If the wiring resistance is only 0.1 Ω, and the motor suddenly draws 3 A, then the voltage drop is $0.1 \times 3 = 0.3$ V.

The battery voltage seen by everything else in the circuit instantly falls by that amount, for as long as the surge lasts.

2.3.3 Why the ESP32 Feels It

If both devices share the same supply, the ESP32 experiences the same dip. Instead of receiving 3.3 V, it may briefly receive 3.0 V or 2.8 V. The consequences are the classic symptoms of an unstable rail:

- Random resets.
- Brownout detection triggering.
- Wi-Fi disconnects.
- Crashes.
- Unstable ADC readings.
- Unexpected program behaviour that looks exactly like a software bug.

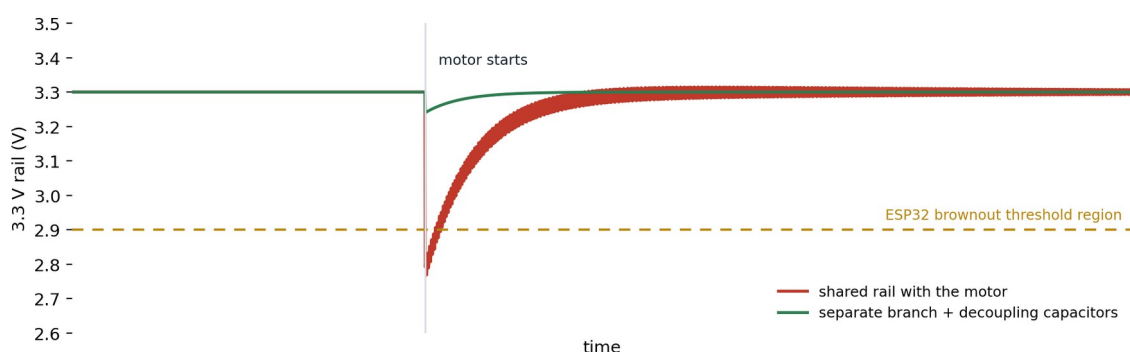


Figure 10. The 3.3 V rail at the instant a motor starts, on a shared rail and on a separate branch with decoupling.

2.3.4 Motors Also Inject Noise

Current is only half of the problem. Motors are inductive loads, and their brushes and windings generate voltage spikes, electromagnetic interference, and switching transients. These disturbances travel along the

power wires into the rest of the system. Even if the voltage never drops far enough to reset the ESP32, this electrical noise can still disturb its operation.

2.3.5 Why the L298N Does Not Solve This

A frequent assumption is: “I am using an L298N, so the ESP32 is protected.” Not exactly. The L298N isolates the control signals from the motor switching logic, but it does not create a clean power supply. The motor still draws current from the supply, and the motor still injects electrical noise into it. If the ESP32 shares the same power rail, it will still experience the voltage dips.

2.3.6 Better Power Distribution

Instead of powering everything from one shared rail, separate the power paths at the source. Both circuits can still use the same battery, but the current paths are kept apart: one branch goes to the motor supply and then to the L298N and motor, and a second branch goes to the voltage regulator and then to the ESP32. The regulator also filters part of the motor’s disturbances before they reach the microcontroller.

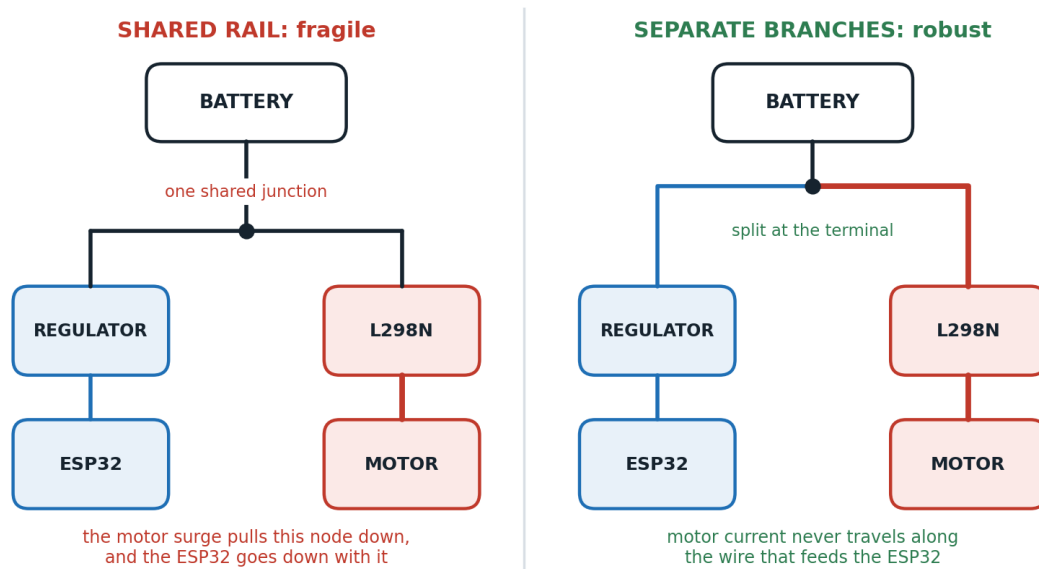


Figure 11. The same battery, wired two different ways. Splitting the branches at the source keeps motor current out of the ESP32’s supply path.

2.3.7 Complete Power Isolation

In industrial systems the logic and motor sides may use completely different power supplies, for example a 5 V logic supply for the controller and a 24 V supply for the motor driver, with an optocoupler passing the commands between them. The result is no shared power rail, no shared electrical noise, and no motor current anywhere near the logic supply. This provides the highest level of protection available.

2.3.8 Do They Still Need a Common Ground?

It depends, and the answer is worth memorising because it catches people out.

- **Without optocouplers:** yes. A common ground is required, because a GPIO voltage is only meaningful relative to a shared 0 V reference. Without it, HIGH and LOW have no defined meaning to the receiving device.
- **With optocouplers:** no. There is no electrical connection at all, so the two systems can use completely separate grounds and completely separate power supplies.

2.3.9 Capacitors Help Too

Even when using the same battery, capacitors placed close to the ESP32 act as small local energy reservoirs. When the motor briefly causes the voltage to dip, the capacitor supplies current for a short time and helps the ESP32 maintain a stable voltage. Large electrolytic capacitors help with slower voltage drops, while small ceramic capacitors filter high-frequency noise. A common pairing near the ESP32 power pins is 100 nF ceramic together with 100 μ F electrolytic. These improve stability, but they do not replace proper power distribution.

2.3.10 A Common Misconception

Many beginners think: “if the battery can provide enough current, everything is fine.” Current capacity is only part of the story. Even a powerful battery cannot eliminate wire resistance, voltage drops, motor-generated electrical noise, ground bounce, or switching transients. Those are the real reasons engineers separate motor and logic power.

2.3.11 Design Recommendations

For a small ESP32 and L298N robot: use the same battery, but split it into separate power branches at the battery terminal, keep a common ground, and place decoupling capacitors near the ESP32. This is the standard and recommended design for hobby-scale work.

For industrial motor control: use a separate 5 V logic supply for the ESP32, pass commands through an optocoupler, and run the motor driver from its own 24 V supply. Separate supplies, no shared electrical power path, and maximum protection against noise and faults.

Question	Answer
Can the ESP32 and motor use the same battery?	Yes, if the power distribution is designed correctly
Should they share the exact same power rail?	Preferably not, because motor current and noise can disturb the ESP32
Why are motors problematic?	They draw large, rapidly changing currents and generate electrical noise
Does the L298N isolate the power supply?	No. It controls the motor but does not clean the supply voltage
What helps protect the ESP32?	Separate power paths, voltage regulators, decoupling capacitors, and isolated supplies where needed
Is a common ground still needed?	Yes if there is no optocoupler. No if full galvanic isolation is used

FINAL MENTAL MODEL

Imagine a house with two appliances. One is a desktop computer. The other is a large power saw. If both are plugged into the same cheap extension cord, every time the saw starts the lights flicker and the computer may reboot. The computer is not damaged because it is connected to the saw. It is affected because they are sharing the same electrical supply.

The motor is the power saw: it draws large bursts of current, creates electrical noise, and causes voltage fluctuations. The ESP32 is the computer: it needs clean, stable power and it can reset or misbehave when the supply becomes noisy.

Golden rule: sharing a battery is usually acceptable. Sharing an unfiltered, noisy power rail is what causes problems. Good designs keep the motor’s high-current path separate from the ESP32’s clean power path, even when both ultimately come from the same battery.

2.4 Section 2 Recap

- An inductor resists changes in current, so interrupting it produces a large voltage spike known as back EMF.
- A flyback diode sits in parallel with the inductive load and gives that energy a safe path to circulate and dissipate.
- The diode protects the switching device, not the relay or motor itself.
- Most L298N modules already contain the flyback diodes the motor needs, but any relay coil you add still needs its own.
- GPIO wires are not one-way roads: signals travel one way, faults travel both ways.
- An optocoupler isolation board is the point where electrical continuity between logic and power ends.
- Hobby-scale L298N projects usually do not need one; industrial, noisy, or separately powered systems do.
- A motor can pull several amps at startup or stall, dragging a shared rail down and browning out the ESP32.
- Split the power branches at the battery terminal, regulate the logic side separately, and add decoupling capacitors.

Section 3: EMI, The Practical Minimum

If you have ever experienced an ESP32 that resets at random, Wi-Fi that disconnects the moment the motor starts, sensor readings that make no sense, MQTT messages that stop arriving, or glitches that vanish as soon as the motor is unplugged, then you have already met electromagnetic interference, whether or not anyone called it that at the time.

Many beginners assume EMI only exists in factories with enormous motors. In reality, even a small brushed DC motor generates enough electrical noise to disturb a microcontroller sitting a few centimetres away. This section explains what EMI is, where it comes from in an ESP32 and L298N project specifically, how it travels, and the three wiring practices that fix most of it at this scale.

3.1 What EMI Is

Electromagnetic interference is unwanted electrical noise that interferes with the normal operation of electronic circuits.

IN PLAIN TERMS

Think of two people having a conversation in a quiet room. Communication is easy.

Now imagine someone starts using a jackhammer beside them. The message is still being sent, exactly as before, but the noise makes it difficult to understand. Electronic circuits experience the same problem, and they have far less ability to ask you to repeat yourself.

3.2 Where EMI Comes From in This Project

An ESP32, L298N, and brushed DC motor system contains several sources of electrical noise. Three of them matter.

3.2.1 Source 1: DC Motor Brush Arcing

A brushed motor contains brushes, a commutator, and rotating windings. As the motor spins, the brushes constantly make and break electrical contact with the commutator. Every time contact is broken while current is flowing, a tiny electrical spark, an arc, forms.

Each of those tiny sparks produces high-frequency electrical noise, voltage spikes, and electromagnetic radiation. Individually they are trivial. But the process repeats hundreds or thousands of times every second, which turns the motor into a continuous small noise generator sitting inside your project.

3.2.2 Source 2: Motor Current Switching by the L298N

The L298N rapidly switches current through the motor. When PWM is used for speed control, the driver continuously turns the motor current on and off, thousands of times every second. Every switching event generates a small electrical disturbance. This is completely normal and unavoidable, and it is a genuine contributor to the overall EMI picture.

3.2.3 Source 3: Relay or Contactor Switching

If you added a relay to disconnect the motor power, as recommended in Section 1.8.5, that relay becomes a third source. When the relay opens, current through an inductive load stops suddenly, and the collapsing magnetic field creates voltage spikes, electrical noise, and small arcs between the relay contacts themselves. Those arcs are another EMI source, which is one more reason a relay coil deserves the flyback protection described in Section 2.1.

3.3 How EMI Travels

Many beginners assume EMI only travels through the air. In fact it has two distinct paths, and they need different countermeasures.

- **Conducted EMI.** The noise travels along wires. The motor injects electrical noise directly into the power supply, which carries it to every other device connected to that supply. This is by far the most common problem in hobby projects.
- **Radiated EMI.** The noise travels through the air. Motor wires carrying rapidly changing current behave like small antennas, and nearby wires pick up that electromagnetic energy.

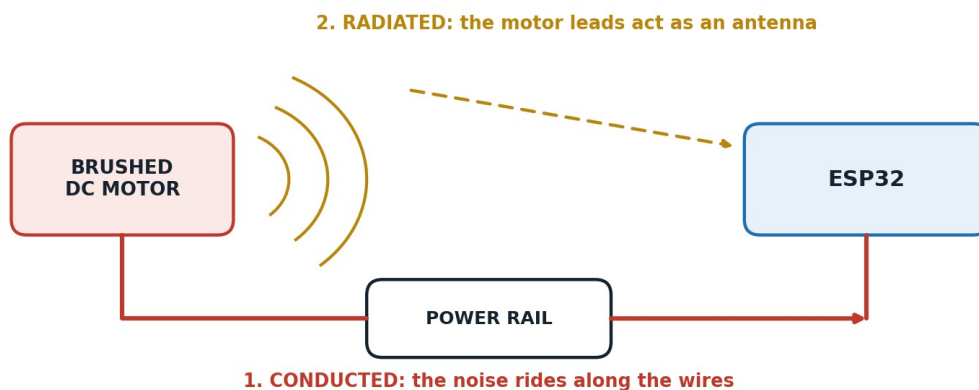


Figure 12. The two routes noise takes from a brushed motor to the ESP32: along the wires, and through the air.

3.4 Symptoms Worth Recognising

EMI rarely announces itself. It shows up disguised as a software problem, which is why so many hours are wasted debugging perfectly correct code. Common symptoms include:

- The ESP32 resetting at random.
- Brownout warnings in the serial monitor.
- Wi-Fi disconnects and dropped MQTT connections.
- False button presses.
- Incorrect sensor readings and encoder errors.
- Random software crashes with no reproducible cause.

A USEFUL DIAGNOSTIC HABIT

If a fault appears only when the motor is running, and disappears when the motor is disconnected, stop reading the code. The problem is electrical.

3.5 The Three Mitigations That Matter at This Scale

There are dozens of professional EMI techniques used in industrial electronics. For a typical embedded project built from an ESP32, an L298N, a brushed DC motor, some sensors, and Wi-Fi or MQTT, only three wiring practices provide the biggest improvement for the least effort and cost:

17. **Single-point grounding, also called star ground.**
18. **Twisted-pair wiring.**

19. Keeping signal cables away from motor cables.

These are simple, inexpensive, and solve most EMI problems in hobby and small embedded systems. It is worth understanding why each one works, because a technique you understand is a technique you will apply correctly.

3.5.1 Single-Point Grounding (Star Ground)

The problem. Every device in the system needs a ground reference, and beginners frequently connect grounds in whatever order is physically convenient: battery to ESP32, ESP32 to L298N, L298N to motor. That chain means the motor's return current flows through the same ground wire the ESP32 is using as its 0 V reference.

Why that is bad. Ground wires have a small but non-zero resistance. When the motor draws 3 A through a shared ground wire, a small voltage appears along that wire, and the ESP32's idea of "zero volts" is no longer perfectly stable. This effect is called ground bounce, and it causes ADC measurement errors, communication problems, random resets, and logic glitches.

The solution. Instead of chaining grounds, connect every device back to one common point, normally the battery negative terminal. The motor current returns directly to the battery, the ESP32 current returns directly to the battery, and the sensor current returns directly to the battery. They no longer share long ground paths. It is called a star ground because every ground wire spreads out from a single central point, like the points of a star.

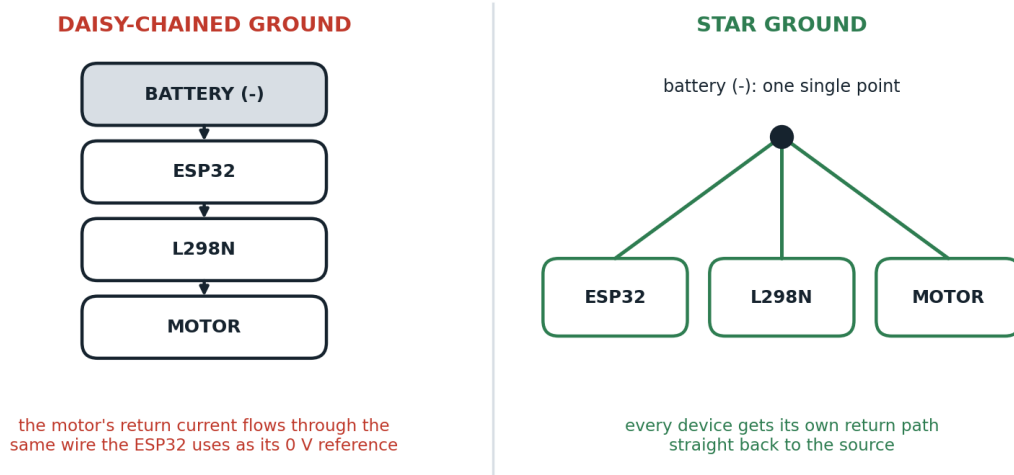


Figure 13. Daisy-chained grounds force the motor's return current through the ESP32's reference. A star ground gives each device its own return path.

3.5.2 Twisted-Pair Wiring

The problem. Motor wires carry rapidly changing current, and changing current creates changing magnetic fields. When the two motor wires run separately, the loop between them is large, and the field spreads out into the surrounding area where nearby wires can pick it up. Straight wires act like antennas.

The solution. Twist the two wires together. Now the magnetic fields from each wire are very close together, and because the currents in the pair flow in opposite directions, their magnetic fields largely cancel each other. Less electromagnetic energy escapes into the environment, and less external noise is picked up.

Why it works. Imagine one wire creating a magnetic field clockwise and the other creating one counter-clockwise. Because the wires are immediately next to each other and swap position along the length of the

cable, the fields overlap and partially cancel. The result is less radiated EMI, less susceptibility to external noise, and more reliable communication.

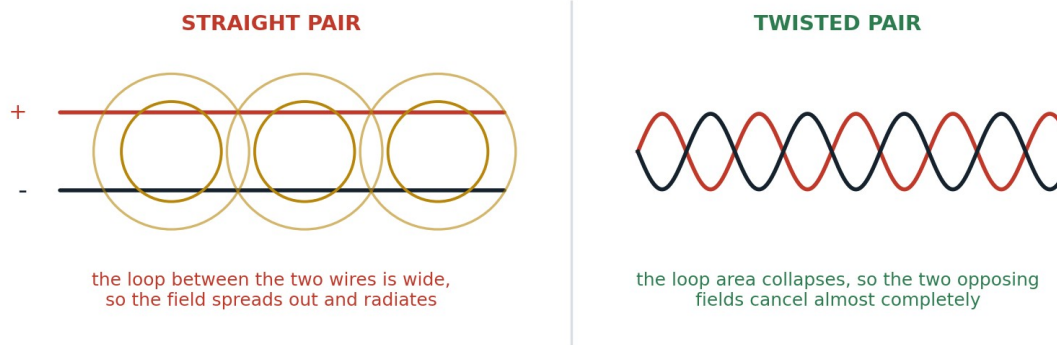


Figure 14. Twisting collapses the loop area between the two conductors so their opposing magnetic fields cancel.

Twisted pairs are useful for motor wires, for differential communication such as CAN and RS-485, and for encoder signals. They are less critical for short GPIO connections on a small PCB, where the loop area is already tiny.

3.5.3 Keeping Signal Cables Away from Motor Cables

The problem. Motor wires carry noisy, high-current signals. Signal wires carry tiny logic voltages. If they run side by side for any distance, the motor cable behaves like a transmitting antenna and the signal cable behaves like a receiving one. Running cables parallel for long distances is the single easiest way to guarantee noise coupling.

The solution. Separate them physically. The greater the distance between the two cables, the weaker the electromagnetic coupling. When they genuinely must cross, cross them at 90 degrees, which reduces the shared length to almost nothing and therefore minimises the coupling.

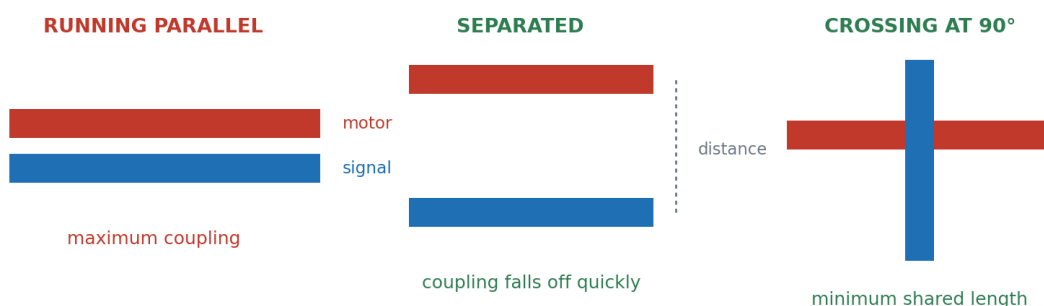


Figure 15. Three cable routing choices, from worst to best: parallel, separated, and crossing at right angles.

3.6 Putting It Together on This Project

For a system of ESP32, then L298N, then a brushed DC motor, a good wiring layout looks like this: the battery negative terminal is the single star ground point, with separate ground returns running from it to the ESP32 and to the L298N; the motor leads are twisted together and kept short; sensor wires and the Wi-Fi antenna are routed well away from the motor wires; and power wires are kept as short as practical.

3.6.1 Common Beginner Mistakes

- Daisy-chaining grounds from battery to ESP32 to L298N to motor.
- Running long motor wires taped alongside sensor cables.
- Leaving motor leads untwisted across the whole chassis.
- Mounting the Wi-Fi antenna directly above the motor.

3.6.2 Practical Checklist

- Use flyback protection wherever an inductive load is switched.
- Place decoupling capacitors, typically 100 nF ceramic plus 100 μ F electrolytic, near the ESP32 power pins.
- Keep motor wires short.
- Twist motor wires together.
- Separate motor wiring from signal wiring, and cross at right angles if they must meet.
- Use a dedicated voltage regulator for the ESP32, fed from its own branch at the battery.
- Add suppression capacitors across a brushed motor if noise becomes a problem: one across the motor terminals, and optionally one from each terminal to the motor casing.

For most hobby robots, these steps solve the large majority of EMI issues without any specialised equipment.

3.6.3 EMI Summary Table

EMI source	Why it creates noise	Typical solution
Brush arcing	Brushes repeatedly make and break contact, creating tiny sparks	Capacitors across the motor, short twisted motor wires
PWM switching	Rapidly switching motor current creates electrical transients	Good layout, decoupling capacitors, short wiring
Relay or contactor switching	Contacts open while current is flowing, creating arcs and voltage spikes	Flyback diodes on relay coils, snubber circuits where appropriate
Long motor wires	They act as antennas, radiating and receiving electromagnetic energy	Keep wires short and twist the motor pair together
Shared power rail	Motor current injects noise directly into the ESP32 supply	Separate power branches and proper voltage regulation

3.6.4 What Each Technique Actually Fixes

Technique	What problem it solves	Why it works
Single-point grounding (star ground)	Ground noise and ground bounce	Prevents large motor currents from flowing through the ESP32's ground path
Twisted-pair wiring	Radiated EMI from current-carrying wires	Opposite currents create magnetic fields that largely cancel each other
Separating signal and motor cables	Noise coupling into GPIO and sensor wires	Reduces the electromagnetic energy transferred between cables

3.7 Do You Need All of This?

For a small robot, you almost certainly do not need industrial-level shielding, isolated DC-DC converters, or EMI filters. But following these three practices will make the system noticeably more reliable for almost no cost. Most unexplained motor-related problems in hobby projects come down to poor grounding, poor cable routing, and motor noise, and these three techniques address exactly those three causes.

FINAL MENTAL MODEL

Imagine your project as a city. Ground wires are the roads returning traffic to the city centre. Motor wires are noisy highways full of heavy trucks. Signal wires are quiet bicycle paths.

If heavy trucks are forced onto the bicycle paths, chaos follows. If the highways run directly beside the bicycle paths, the noise and vibration disturb everyone.

A good engineer designs the city so that every road returns to one central hub, which is the star ground; each highway keeps its traffic tightly together, which is the twisted pair; and highways stay away from quiet neighbourhoods, which is cable separation.

Golden rule: motors are naturally noisy devices. The goal is not to stop them from making noise, it is to keep that noise away from the ESP32, the sensors, and the communication lines, through good wiring, proper power distribution, and simple suppression.

3.8 Section 3 Recap

- EMI is unwanted electrical noise, and in this project it comes mainly from brush arcing, PWM switching, and relay contact arcing.
- It travels two ways: conducted along wires, and radiated through the air.
- Its symptoms look like software bugs, which is why it wastes so much debugging time.
- Star grounding stops motor return current from corrupting the ESP32's 0 V reference.
- Twisted-pair wiring cancels the magnetic field around the motor leads.
- Physical separation, and 90 degree crossings, cut the coupling between motor and signal cables.
- These three practices, plus flyback diodes and decoupling capacitors, are the practical minimum at this scale.

References

Section 1: Relay, Optocoupler, and Solid-State Relay

- Omron Electronic Components. Relays: Basic Knowledge. <https://components.omron.com/us-en/products/basic-knowledge/relays/basics>
- Galco Industrial Electronics. Industrial Control: Relays. <https://www.galco.com/resources/industrial-control/relays>
- Galco Industrial Electronics. How Relays Work (video). <https://www.youtube.com/watch?v=O9uxukyX5Q>
- Electronics Tutorials. The Optocoupler. <https://www.electronics-tutorials.ws/blog/optocoupler.html>
- Build Electronic Circuits. Optocoupler: What It Is and How It Works. <https://www.build-electronic-circuits.com/optocoupler/>
- Electronics Tutorials. Solid State Relay. <https://www.electronics-tutorials.ws/power/solid-state-relay.html>
- GEYA. Optocoupler vs Solid State Relay. <https://www.geya.net/optocoupler-vs-solid-state-relay/>
- Shenler. Solid State Relay vs Mechanical Relay. <https://www.shenler.com/new/solid-state-relay-vs-mechanical/>

Section 2: Protecting the ESP32

- CircuitBread. How Does a Flyback Diode Work? <https://www.circuitbread.com/ee-faq/how-does-a-flyback-diode-work>
- Allelco Electronics. What Is a Flyback Diode, How It Works, and Why Your Circuit Needs One. <https://www.allelcoelec.com/blog/What-Is-a-Flyback-Diode-How-It-Works-and-Why-Your-Circuit-Needs-One.html>
- Flux. Understanding the Flyback Diode: Your Essential Guide to Functionality and Why You Need One. <https://www.flux.ai/p/blog/understanding-the-flyback-diode-your-essential-guide-to-functionality-and-why-you-need-one>
- Flyback diode explained (video). <https://www.youtube.com/watch?v=Ew4Hg4J53gc>

Section 3: Electromagnetic Interference

- GeeksforGeeks. Electromagnetic Interference. <https://www.geeksforgeeks.org/electronics-engineering/electromagnetic-interference/>
- TME. Electromagnetic Interference (EMI): Identification and Prevention Methods in Electronics. <https://www.tme.eu/en/news/library-articles/page/68660/electromagnetic-interference-emi-identification-and-prevention-methods-in-electronics/>

All figures in this report were drawn specifically for it and are original schematic illustrations, not reproductions from the sources listed above.